PayOS - Cổng Thanh Toán Việt Nam VN

📌 Define (Định nghĩa)

PayOS là một **payment gateway** (cổng thanh toán) của Việt Nam, được phát triển bởi **PayOS Vietnam**. Nó hoạt động như một **trung gian xử lý thanh toán** giữa:

- **Merchant** (người bán/doanh nghiệp)
- **Customer** (khách hàng)
- **Banks/Payment providers** (ngân hàng/nhà cung cấp thanh toán)

PayOS cho phép doanh nghiệp chấp nhận thanh toán trực tuyến qua nhiều phương thức: thẻ ATM nội địa, thẻ tín dụng/ghi nợ quốc tế, ví điện tử, QR code, chuyển khoản ngân hàng...

🎯 Why Does It Use? (Tại sao sử dụng?)

1. **Giải quyết bài toán thanh toán online phức tạp**

```
Không có PayOS:
Ứng dụng của bạn → Tích hợp 10+ ngân hàng riêng lẻ
                  → Duy trì 10+ API khác nhau
                  → Xử lý 10+ webhook formats
                  → Compliance 10+ bộ security rules

Có PayOS:
Ứng dụng của bạn → PayOS → 10+ ngân hàng
                  ↓
            Chỉ 1 API, 1 webhook format!
```

2. **Tập trung vào business logic**

- Không cần hiểu sâu về payment processing
- Không cần deal với từng ngân hàng
- Không cần lo về PCI-DSS compliance

3. **Tối ưu chuyển đổi thanh toán**

- UI/UX được tối ưu cho thị trường Việt Nam
- Hỗ trợ nhiều phương thức thanh toán phổ biến
- Xử lý nhanh, giảm tỷ lệ bỏ giỏ hàng

💡 Why Need? (Tại sao cần?)

Kịch bản thực tế:

Bạn đang xây dựng **booking system** (như trong code trên):

❌ KHÔNG dùng Payment Gateway:

```
// Ác mộng bắt đầu...
async createBooking() {
  // 1. Khách hàng book phòng
  const booking = await Booking.create({...});

  // 2. Làm sao nhận tiền??? 😱
  // - Cho số tài khoản → Khách chuyển khoản thủ công
  // - Nhân viên check bank statement
  // - Gọi điện xác nhận "Anh đã chuyển tiền chưa?"
  // - Cập nhật DB thủ công
  // - GỬI EMAIL XÁC NHẬN THỦ CÔNG

  // → Mất 30 phút - 2 giờ / booking
  // → Sai sót cao
  // → Khách hàng bỏ đi
}
```

☑ CÓ PayOS:

```
async createBooking() {
  const booking = await Booking.create({...});

  // Tạo link thanh toán tự động
  const paymentLink = await paymentService.createPaymentLink(booking.id);

  // Khách thanh toán → Webhook tự động → Email tự động
  // → 2-5 phút hoàn tất
  // → 100% chính xác
  // → Khách hài lòng
}
```

🕐 When Need? (Khi nào cần?)

☑ CẦN PayOS khi:

1. **Bán hàng/dịch vụ online:**
    - E-commerce
    - Booking systems (hotel, tour, spa...)
    - SaaS subscriptions
    - Course platforms
2. **Yêu cầu thanh toán tức thì:**

- Không muốn khách chuyển khoản thủ công
- Cần xác nhận thanh toán real-time
- Tự động hóa quy trình

3. **Có khách hàng cuối (end-users):**
   - B2C businesses
   - Marketplace platforms

❌ KHÔNG cần PayOS khi:

1. **B2B với thanh toán sau:**
   - Khách doanh nghiệp → xuất hóa đơn → thanh toán 30-60 ngày
2. **Dịch vụ miễn phí:**
   - Free tier applications
   - Internal tools
3. **Thanh toán offline:**
   - Cửa hàng vật lý dùng POS machine

🔄 When Use? (Khi nào sử dụng trong code?)

Payment Flow trong ứng dụng:

```
// 1️⃣ KHÁCH TẠO BOOKING (Chưa thanh toán)
POST /bookings
{
  roomId: "123",
  checkIn: "2025-12-01",
  checkOut: "2025-12-05"
}

Response: {
  bookingId: "abc-xyz",
  status: "pending",  // ← Chờ thanh toán
  totalAmount: 5000000
}

// 2️⃣ TẠO PAYMENT LINK (Gọi PayOS)
POST /payments/create
{
  bookingId: "abc-xyz"
}

Response: {
  checkoutUrl: "https://payos.vn/checkout/ORDER-123456",
  orderCode: "ORDER-123456"
```

```
}

// 3️⃣ KHÁCH THANH TOÁN
// User click vào checkoutUrl → Trang PayOS
// Chọn phương thức → Nhập thông tin → Xác nhận
// PayOS xử lý với ngân hàng

// 4️⃣ PAYOS GỬI WEBHOOK (Tự động)
POST /payments/webhook  // ← PayOS gọi
{
  code: "00",  // Success
  orderCode: "ORDER-123456",
  amount: 5000000
}

// Server tự động:
// - Update booking status: pending → confirmed
// - Gửi email xác nhận
// - Cập nhật payment.paidAt

// 5️⃣ CHECK STATUS (Optional)
GET /payments/status/ORDER-123456

Response: {
  payment: { status: "PAID" },
  booking: { status: "confirmed" }
}
```

🎬 Use Case Usage (Các tình huống sử dụng)

Use Case 1: Hotel Booking System ⭐ (Như code của bạn)

```
// Khách đặt phòng khách sạn
const booking = await bookingService.create({
  userId: "user123",
  roomId: "deluxe-101",
  checkIn: "2025-12-25",
  checkOut: "2025-12-27",
  totalAmount: 3000000
});

// Tạo link thanh toán
const payment = await paymentService.createPaymentLink(booking.id);

// Gửi link cho khách
await emailService.send({
  to: booking.email,
  subject: "Xác nhận đặt phòng - Thanh toán",
  body: `Click để thanh toán: ${payment.checkoutUrl}`
});
```

```
// Webhook tự động xử lý khi khách thanh toán xong
```

Use Case 2: E-Commerce

```
// Khách mua hàng
const order = await Order.create({
  items: [
    { product: "iPhone 15", price: 25000000, qty: 1 }
  ],
  total: 25000000
});

// Redirect đến PayOS
const paymentLink = await payOS.createPaymentLink({
  orderCode: order.id,
  amount: order.total,
  description: `Đơn hàng ${order.id}`,
  returnUrl: "https://shop.com/order-success",
  cancelUrl: "https://shop.com/order-cancel"
});

// Webhook → Gửi hàng khi nhận được tiền
```

Use Case 3: SaaS Subscription

```
// Khách nâng cấp gói Pro
const subscription = await Subscription.create({
  userId: "user456",
  plan: "PRO",
  price: 500000,
  period: "monthly"
});

// Thanh toán định kỳ hàng tháng
const payment = await payOS.createPaymentLink({
  amount: 500000,
  description: "Gói Pro - Tháng 12/2025"
});

// Webhook → Active tính năng Pro
```

⚠ Popular Pitfalls (Lỗi phổ biến)

PITFALL #1: Không xử lý webhook đúng cách

❌ **SAI:**

```
async handleWebhook(req: Request, res: Response) {
  const { code, orderCode } = req.body;

  // Chỉ check code = "00" → Dễ bị fake request!
  if (code === "00") {
    await Booking.updateOne(
      { orderCode },
      { status: "confirmed" }
    );
  }

  res.json({ success: true });
}
```

Vấn đề: Hacker có thể gửi POST request giả mạo!

☑ **ĐÚNG:**

```
async handleWebhook(req: Request, res: Response) {
  // 1. Verify webhook signature từ PayOS
  const signature = req.headers['x-payos-signature'];
  const isValid = payOS.verifyWebhookSignature(
    req.body,
    signature,
    config.payos.checksumKey
  );

  if (!isValid) {
    return res.status(401).json({ error: "Invalid signature" });
  }

  // 2. Check với PayOS API để double-confirm
  const paymentInfo = await payOS.getPaymentLinkInformation(orderCode);

  if (paymentInfo.status !== "PAID") {
    return res.status(400).json({ error: "Payment not confirmed" });
  }

  // 3. Mới update database
  await Booking.updateOne({ orderCode }, { status: "confirmed" });

  res.json({ success: true });
}
```

Resolution:

- ☑ Verify webhook signature

- ☑ Double-check với PayOS API
- ☑ Implement idempotency (tránh xử lý 2 lần)

PITFALL #2: Không handle payment timeout

❌ SAI:

```
// User tạo payment link → Bỏ không thanh toán
// Booking vẫn ở trạng thái "pending" mãi mãi
```

☑ ĐÚNG:

```
// Cron job chạy mỗi 30 phút
async cleanupExpiredPayments() {
  const expiredBookings = await Booking.find({
    status: "pending",
    createdAt: { $lt: new Date(Date.now() - 30 * 60 * 1000) }
  });

  for (const booking of expiredBookings) {
    // Check với PayOS xem có thanh toán không
    try {
      const payment = await payOS.getPaymentLinkInformation(booking.orderCode);

      if (payment.status === "PAID") {
        // Late webhook → Update ngay
        await this.handlePaymentSuccess(booking);
      } else {
        // Chưa thanh toán → Cancel booking
        await Booking.updateOne(
          { _id: booking.id },
          { status: "cancelled", cancelReason: "Payment timeout" }
        );
      }
    } catch (error) {
      // Payment link không tồn tại → Cancel
      await Booking.updateOne(
        { _id: booking.id },
        { status: "cancelled" }
      );
    }
  }
}
```

Resolution:

- ☑ Set payment expiry time (15-30 phút)
- ☑ Cron job cleanup

- ☑ Notify user về cancellation

PITFALL #3: Không handle concurrent payments

❌ SAI:

```
// User click "Thanh toán" nhiều lần → Tạo nhiều payment links
// → Có thể thanh toán 2 lần cho cùng booking!
```

☑ ĐÚNG:

```
async createPaymentLink(bookingId: string) {
  const booking = await Booking.findById(bookingId);

  // 1. Check nếu đã có payment link active
  if (booking.payment?.transactionId) {
    const existingPayment = await payOS.getPaymentLinkInformation(
      booking.payment.transactionId
    );

    if (existingPayment.status === "PENDING") {
      // Trả về link cũ thay vì tạo mới
      return {
        checkoutUrl: existingPayment.checkoutUrl,
        orderCode: existingPayment.orderCode
      };
    }
  }

  // 2. Dùng database lock
  const updated = await Booking.findOneAndUpdate(
    {
      _id: bookingId,
      status: "pending",
      "payment.status": { $ne: "paid" }  // Chưa thanh toán
    },
    {
      $set: {
        "payment.status": "processing"  // Lock
      }
    }
  );

  if (!updated) {
    throw new Error("Booking already being processed or paid");
  }

  // 3. Tạo payment link
  const payment = await payOS.createPaymentLink({...});
```

```
  return payment;
}
```

Resolution:

- ☑ Check existing payment links
- ☑ Database-level locking
- ☑ Idempotency keys

PITFALL #4: Amount mismatch

❌ SAI:

```
// Tạo booking: 5,000,000 VND
const booking = await Booking.create({ amount: 5000000 });

// Nhưng tạo payment: 50,000 VND (Thiếu 2 số 0!)
await payOS.createPaymentLink({
  amount: 50000  // ← Bug!
});

// → Khách thanh toán thiếu → Không nhận phòng được
```

☑ ĐÚNG:

```
async createPaymentLink(bookingId: string) {
  const booking = await Booking.findById(bookingId);

  // Validate amount
  if (!booking.totalAmount || booking.totalAmount <= 0) {
    throw new Error("Invalid booking amount");
  }

  // Double-check calculation
  const calculatedAmount = this.calculateBookingAmount(booking);
  if (calculatedAmount !== booking.totalAmount) {
    throw new Error("Amount mismatch detected");
  }

  // Tạo payment với exact amount
  const payment = await payOS.createPaymentLink({
    amount: booking.totalAmount,  // ← Đúng
    description: `Booking ${bookingId} - ${booking.totalAmount.toLocaleString()}
VND`
  });

  // Lưu amount vào payment record để verify sau
```

```
  await Booking.updateOne(
    { _id: bookingId },
    {
      "payment.amount": booking.totalAmount,
      "payment.transactionId": payment.orderCode
    }
  );

  return payment;
}

// Trong webhook
async handleWebhook(data) {
  const booking = await Booking.findOne({
    "payment.transactionId": data.orderCode
  });

  // VERIFY AMOUNT!
  if (data.amount !== booking.payment.amount) {
    logger.error("Amount mismatch in webhook", {
      expected: booking.payment.amount,
      received: data.amount
    });
    throw new Error("Payment amount mismatch");
  }

  // Proceed...
}
```

Resolution:

- ☑ Validate amounts at multiple points
- ☑ Store expected amount in DB
- ☑ Verify in webhook
- ☑ Use integer for money (avoid float: 5000000, not 5000000.00)

PITFALL #5: Không test thoroughly

❌ SAI:

```
// Chỉ test success case
// Deploy lên production → Fail cases crash app
```

☑ ĐÚNG:

```
// Test cases cần cover:
describe("PayOS Integration", () => {
  it("should create payment link successfully", async () => {
```

```
    // Happy path
  });

  it("should handle PayOS API down", async () => {
    // PayOS unreachable → Graceful error
  });

  it("should handle duplicate webhook calls", async () => {
    // Idempotency test
  });

  it("should handle late webhook arrival", async () => {
    // Cron job picks up → Manual check
  });

  it("should prevent double payment", async () => {
    // Concurrent payment attempts
  });

  it("should handle amount mismatch", async () => {
    // Webhook amount != booking amount
  });

  it("should handle payment expiry", async () => {
    // 30 minutes timeout
  });

  it("should verify webhook signature", async () => {
    // Fake webhook rejected
  });
});
```

Resolution:

- ☑ Test với PayOS sandbox environment
- ☑ Mock webhook calls trong tests
- ☑ Load testing với concurrent requests
- ☑ Monitor production payments closely lúc đầu

Summary

Aspect	Key Points
What	Payment gateway của VN, trung gian xử lý thanh toán
Why	Đơn giản hóa payment integration, 1 API thay vì 10+
When Need	Bán hàng online, cần thanh toán tức thì, B2C
When Use	Create booking → Generate link → User pays → Webhook → Confirm
Pitfalls	Webhook security, timeouts, concurrency, amount validation

Key Takeaway: PayOS giúp bạn tập trung vào business logic thay vì làm payment engineer! 🚀